Resumen de estudio: Kotlin en Android (examen tipo test)

Temas: View Binding, RecyclerView, Intents y filtros, actividades e intents (codelab Words), compartir datos y ActivityResultLauncher.

1. View Binding

Qué es: una función de Jetpack que genera una *clase de binding* por cada layout XML del módulo, con referencias directas a las vistas que tienen `id`. Sustituye a `findViewById`.

Activación (por módulo), en `build.gradle`:

```
android {
    buildFeatures {
        viewBinding = true
    }
}
```

Nombre de la clase generada: nombre del XML en Pascal case + `Binding`.

Layout	Clase generada
`result_profile.xml`	`ResultProfileBinding`
`activity_main.xml`	`ActivityMainBinding`

Detalles importantes:

- Solo se generan campos para las vistas con `android:id`. Un `ImageView` sin id no aparece.
- Cada clase tiene `getRoot()` (en Kotlin, `binding.root`), que devuelve la vista raíz del layout.
- Para ignorar un layout: `tools:viewBindingIgnore="true"` en la vista raíz.
- Si un id tiene tipos distintos según la configuración (por ejemplo `TextView` en `layout/` y `EditText` en `layout-land/`), el campo se genera como `View`. Se soluciona con `tools:viewBindingType="TextView"`. El valor debe heredar de `View` y ser superclase de la etiqueta.

En una Activity (en `onCreate`):

```
private lateinit var binding: ResultProfileBinding

override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    binding = ResultProfileBinding.inflate(layoutInflater)
    setContentView(binding.root)
}
```

Orden: `inflate()` → obtener `root` → `setContentView(root)`.

En un Fragment (en `onCreateView` / `onDestroyView`):

```
private var _binding: ResultProfileBinding? = null
private val binding get() = _binding!!

override fun onCreateView(inflater: LayoutInflater, container: ViewGroup?,
savedInstanceState: Bundle?): View? {
    _binding = ResultProfileBinding.inflate(inflater, container, false)
    return binding.root
}

override fun onDestroyView() {
    super.onDestroyView()
    _binding = null
}
```

- Se pone `_binding = null` en `onDestroyView()` porque **los fragments sobreviven a sus vistas** (evita fugas de memoria).
- Si el layout ya está inflado se usa `bind()` en lugar de `inflate()`.

Ventajas frente a `findViewById`:

- **Seguridad contra nulos:** referencias directas, sin ids inválidos. Si una vista solo existe en algunas configuraciones, el campo se marca `@Nullable`.
- **Seguridad de tipos:** los campos tienen el tipo real de la vista, así que no hay `ClassCastException`.
- Los errores salen **al compilar**, no en ejecución.

View Binding vs Data Binding:

	View Binding	Data Binding
Compilación	Más rápida (sin procesamiento de anotaciones)	Más lenta

	View Binding	Data Binding
Uso	Se aplica automáticamente a todos los layouts del módulo	Requiere layouts etiquetados especialmente
Variables y expresiones en el XML	No	Sí
Binding bidireccional	No	Sí

Se pueden usar los dos en un mismo proyecto.

2. RecyclerView

Idea clave: muestra grandes conjuntos de datos de forma eficiente. Cuando un elemento sale de pantalla, **no se destruye: se reutiliza (recicla)** para los nuevos elementos. Mejora rendimiento y consumo de energía.

Clases clave:

Clase	Función
`RecyclerView`	El `ViewGroup` que contiene las vistas de los datos. Se añade al layout como cualquier vista
`ViewHolder` (extiende `RecyclerView.ViewHolder`)	Wrapper alrededor de la `View` de un elemento. Al crearse no tiene datos
`Adapter` (extiende `RecyclerView.Adapter`)	Crea los holders y los vincula con los datos
`LayoutManager`	Organiza los elementos en pantalla

Métodos que se sobrescriben en el Adapter:

1. `onCreateViewHolder()`: crea e inicializa el `ViewHolder` y su `View`, **pero no rellena los datos**.
2. `onBindViewHolder()`: asocia el holder con los datos (vinculación / *binding*), por ejemplo `holder.textView.text = dataSet[position]`.
3. `getItemCount()`: devuelve el tamaño del conjunto de datos.

LayoutManagers incluidos:

- `LinearLayoutManager`: lista unidimensional.
- `GridLayoutManager`: cuadrícula 2D. En vertical, los elementos de una fila tienen el mismo ancho y alto, pero las filas pueden tener alturas distintas.

- `StaggeredGridLayoutManager`: como el anterior, pero **no exige** misma altura en una fila (vertical) ni mismo ancho en una columna (horizontal).

Pasos de implementación:

1. Decidir cómo se ve la lista (LayoutManager).
2. Diseñar el layout de cada elemento y crear el `ViewHolder`.
3. Definir el `Adapter`.

Uso típico:

```
val recyclerView: RecyclerView = findViewById(R.id.recycler_view)
recyclerView.layoutManager = LinearLayoutManager(this)
recyclerView.adapter = CustomAdapter(dataset)
```

Pantalla de borde a borde: `enableEdgeToEdge()`, aplicar insets al `RecyclerView` (con `fitsSystemWindows="true"` o `setOnApplyWindowInsetsListener`) y `android:clipToPadding="false"`.

3. Intents y filtros de intents

Definición: un `Intent` es un objeto de mensajería para pedir una acción a otro componente de la app.

Tres usos principales:

1. **Iniciar una Activity:** `startActivity(intent)`.
2. **Iniciar un Service:** `startService()` / `bindService()` (o `JobScheduler` desde Android 5.0).
3. **Enviar un broadcast:** `sendBroadcast()` / `sendOrderedBroadcast()`.

Explícito vs implícito

	Explícito	Implícito
Qué indica	El componente exacto (`ComponentName`)	Una acción general a realizar
Uso típico	Navegar dentro de tu propia app	Pedir algo a otras apps (mapa, compartir, abrir enlace)
Ejemplo	`Intent(this, DetailActivity::class.java)`	`Intent(Intent.ACTION_VIEW, uri)`

- Sin nombre de componente, el intent es implícito.

- Un intent explícito **siempre** se entrega, sin importar los filtros del destino.
- Si hay varias apps compatibles con un implícito, el sistema muestra un diálogo de selección.
- Si ninguna app puede atenderlo, se lanza `ActivityNotFoundException` (conviene capturarla).
- **Para un `Service` se usa siempre intent explícito.** Desde Android 5.0, `bindService()` con intent implícito lanza excepción.

Qué contiene un Intent

- **Nombre del componente:** lo que lo hace explícito.
- **Acción:** cadena con la acción genérica (`ACTION_VIEW` , `ACTION_SEND` , `ACTION_DIAL` , `ACTION_EDIT` ...). Las acciones propias llevan el nombre del paquete como prefijo.
- **Datos:** URI (`Uri`) y/o tipo MIME.
- **Categoría:** información extra sobre qué tipo de componente debe atenderlo (`CATEGORY_LAUNCHER` , `CATEGORY_BROWSABLE`).
- **Extras:** pares clave-valor (`putExtra()` , o un `Bundle` con `putExtras()`).
- **Flags:** metadatos sobre cómo iniciar la actividad (tarea, historial...).

Importante sobre datos y MIME:

- `setData()` fija solo el URI, `setType()` fija solo el MIME, y **cada uno anula el valor del otro**.
- Para fijar ambos hay que usar **`setDataAndType()`** .
- Un URI `content:` indica que los datos los gestiona un `ContentProvider` .

Extras: no usar `Parcelable` ni `Serializable` en intents pensados para otras apps (pueden provocar `RuntimeException` si la otra app no tiene la clase).

Filtros de intents (`<intent-filter>`)

Se declaran en el **manifest**, dentro del componente. Admiten `<action>` , `<category>` y `<data>` .

```
<activity android:name="ShareActivity" android:exported="false">
    <intent-filter>
        <action android:name="android.intent.action.SEND"/>
        <category android:name="android.intent.category.DEFAULT"/>
        <data android:mimeType="text/plain"/>
    </intent-filter>
</activity>
```

Reglas que suelen preguntar:

- Para recibir intents implícitos **es obligatorio** incluir `CATEGORY_DEFAULT` en el filtro, porque `startActivity()` y `startActivityForResult()` la añaden automáticamente a los implícitos.
- `MAIN` + `LAUNCHER` hace que la actividad sea el punto de entrada y aparezca en el lanzador de apps. **Ambos deben ir juntos.**
- Los valores de `<action>` y `<category>` son la cadena literal, no la constante de la clase.
- Si una Activity **no** declara filtros, solo se puede iniciar con intent explícito.
- Un intent debe pasar **las tres pruebas** (acción, categoría y datos) para llegar al componente.
- Un componente puede tener varios filtros, y basta con que el intent pase uno.
- **Android 12+:** si un componente tiene intent-filter, hay que declarar `android:exported` de forma explícita, o la app **no se instala**. Es `true` para el launcher y `false` por seguridad en el resto.
- Un filtro **no es** una forma segura de impedir que otras apps inicien tus componentes. Para eso, `exported="false"`.
- Los filtros de actividades van siempre en el manifest, pero los de **BroadcastReceiver** se pueden registrar dinámicamente con `registerReceiver()` / `unregisterReceiver()`.

Prueba de acción: el intent debe coincidir con alguna acción del filtro. Si el filtro no tiene acciones, todo falla. Si el intent no tiene acción, pasa siempre que el filtro tenga al menos una.

Prueba de categoría: cada categoría del intent debe estar en el filtro. El filtro puede tener más. Un intent sin categorías siempre pasa.

Prueba de datos (URI = `scheme://host:port/path`):

- Cada parte es opcional, pero hay dependencias: sin scheme se ignora el host, sin host se ignora el puerto, y sin scheme ni host se ignora el path.
- Si el filtro solo lista un tipo MIME, se asume que acepta datos `content:` y `file:`.
- Si el intent tiene URI o MIME y el filtro no tiene ningún `<data>`, la prueba falla.

Android 13+: los intents de apps externas solo llegan a un componente exportado si coinciden con acción y categoría de algún filtro. No se aplica a intents de la misma app, de la propia, del sistema o de root, ni a componentes sin filtros.

PackageManager: `queryIntentActivities()` devuelve las actividades que pueden atender un intent (no las activa). Hay equivalentes para servicios y receptores.

PendingIntent

- Es un wrapper de un `Intent` que da permiso a otra app (o al sistema) para ejecutarlo **como si fuera tu app**.
- Casos de uso: notificaciones (`NotificationManager`), widgets y alarmas (`AlarmManager`).
- Creadores: `getActivity()` , `getService()` , `getBroadcast()` .
- **Android 12+:** hay que indicar la mutabilidad, con `FLAG_IMMUTABLE` o `FLAG_MUTABLE` . Si no, lanza `IllegalArgumentException` . Se recomienda `FLAG_IMMUTABLE` salvo casos concretos: respuesta directa en notificaciones, burbujas, `AlarmManager` , actualizaciones de ubicación y Android Auto.
- Buena práctica: envolver siempre un intent **explícito**.

Seguridad (Android 12+):
`StrictMode.VmPolicy.Builder().detectUnsafeIntentLaunch()` detecta el lanzamiento inseguro de un *intent anidado* (extraído de los extras de otro intent y lanzado enseguida). Para evitarlo: copiar solo los extras necesarios, no exportar componentes innecesarios y usar `PendingIntent` en lugar de un intent anidado.

4. Codelab: Intents y actividades (app Words)

Idea de la app: `MainActivity` muestra letras en un `RecyclerView` (`LetterAdapter`). Al pulsar una letra se abre `DetailActivity` con las palabras de esa letra (`WordAdapter`). Al pulsar una palabra se abre el navegador con su definición.

Intent explícito (letra → `DetailActivity`), en `LetterAdapter.onBindViewHolder()` :

```
holder.button.setOnClickListener {
    val context = holder.itemView.context
    val intent = Intent(context, DetailActivity::class.java)
    intent.putExtra(DetailActivity.LETTER, holder.button.text.toString())
    context.startActivity(intent)
}
```

- Se usa `toString()` porque `text` es un `CharSequence` y `putExtra()` espera `String` .
- Un *extra* es un dato con nombre, como un argumento al llamar a una función.

Recoger el extra en `DetailActivity.onCreate()` :

```
val letterId = intent?.extras?.getString(LETTER).toString()
```

- `intent` es una propiedad de cualquier Activity (el intent que la inició).
- `extras` es un `Bundle` .

- Ambos admiten `null`, por eso se usa `?.` (llamada segura).
- `getString()` devuelve `String?`, y `toString()` garantiza una `String` no nula.

Companion object (para constantes compartidas):

```
companion object {
    const val LETTER = "letter"
    const val SEARCH_PREFIX = "https://www.google.com/search?q="
}
```

- Solo hay una instancia del `companion object` (patrón singleton) y está asociado a la clase, no a una instancia.
- Se accede con `DetailActivity.LETTER`.
- Evita repetir *strings* "hard-coded".

Intent implícito (palabra → navegador), en `WordAdapter.onBindViewHolder()`:

```
holder.button.setOnClickListener {
    val queryUrl: Uri = Uri.parse("${DetailActivity.SEARCH_PREFIX}${item}")
    val intent = Intent(Intent.ACTION_VIEW, queryUrl)
    context.startActivity(intent)
}
```

- No se nombra ninguna actividad: el sistema decide qué app abre el URI.
- URI = identificador uniforme de recursos. Todas las URL son URI, pero no todos los URI son URL (por ejemplo `tel:`).
- Otras acciones o categorías mencionadas: `ACTION_SET_ALARM`, `ACTION_DIAL`, `CATEGORY_APP_MAPS`, `CATEGORY_APP_EMAIL`, `CATEGORY_APP_GALLERY`.

Menú de opciones (alternar lista/cuadrícula):

- Archivo de menú XML en `res/menu` (por ejemplo `layout_menu.xml`) con un `<item>` con `id`, `title`, `icon` y `app:showAsAction="always"` (siempre visible en la barra de la app).
- `onCreateOptionsMenu()`: infla el menú con `menuInflater.inflate(...)` y devuelve `true`.
- `onOptionsItemSelected()`: comprueba qué item se pulsó (con `when` sobre `item.itemId`), alterna `isLinearLayoutManager` y llama a `chooseLayout()` y `setIcon()`.
- `chooseLayout()` asigna `LinearLayoutManager(this)` o `GridLayoutManager(this, 4)` y vuelve a asignar el adaptador.

Pasos para lanzar un intent (resumen del codelab):

1. Obtener una referencia al contexto.
2. Crear el `Intent` (actividad o tipo de intent).
3. `putExtra()` si hacen falta datos.
4. `startActivity(intent)`.

5. Compartir datos simples con otras apps

Dos formas de compartir:

- **Android Sharesheet:** para enviar contenido fuera de tu app o a otra persona. Se muestra con `Intent.createChooser()`.
- **Selector de resolución de intents (intent resolver):** para pasar datos a la siguiente etapa de una tarea bien definida. No se llama a `createChooser()`. Si solo una app coincide, se abre directamente.

Compartir texto:

```
val sendIntent: Intent = Intent().apply {
    action = Intent.ACTION_SEND
    putExtra(Intent.EXTRA_TEXT, "Texto a enviar")
    type = "text/plain"
}
val shareIntent = Intent.createChooser(sendIntent, null)
startActivity(shareIntent)
```

Extras útiles: `EXTRA_TEXT`, `EXTRA_TITLE`, `EXTRA_SUBJECT`, `EXTRA_EMAIL` / `EXTRA_CC` / `EXTRA_BCC`. Estos tres últimos esperan un `String[]`, y se añaden con `putExtra(String, String[])`.

Compartir contenido binario (imágenes, etc.):

- `ACTION_SEND` con MIME adecuado (por ejemplo `image/jpeg`) y un URI en `EXTRA_STREAM`.
- La app receptora necesita permiso de acceso al URI: usar un `ContentProvider` (mejor con `FileProvider` y permisos por URI, que son temporales) o `MediaStore`.

Varios elementos: `ACTION_SEND_MULTIPLE` con `putParcelableArrayListExtra(EXTRA_STREAM, lista)`.

MIME recomendados:

Receptores se registran para	Emisores envían
`text/*`	`text/plain`, `text/html`, `text/rtf`, `text/json`

Receptores se registran para	Emisores envían
`image/*`	`image/jpg`, `image/png`, `image/gif`
`video/*`	`video/mp4`, `video/3gp`

- Dar el MIME **más específico** posible.
- Evitar `*/*` salvo que se compartan varios tipos mezclados.

Por qué usar Sharesheet: mantiene la coherencia entre apps y puede sugerir destinos con información del sistema. No hay que crear tu propia lista de destinos.

Novedades del Sharesheet:

- Vistas previas de texto enriquecido desde **Android 10 (API 29)**, con `EXTRA_TITLE` y una miniatura mediante `ClipData` / `data` y `FLAG_GRANT_READ_URI_PERMISSION`.
- Acciones personalizadas (`ChooserAction`) desde **Android 14 (API 34)**.
- `EXTRA_CHOOSER_TARGETS` y `EXTRA_INITIAL_INTENTS` (se añaden **después** de `createChooser()`), para destinos propios.
- `EXTRA_EXCLUDE_COMPONENTS` para excluir destinos que controlas.
- Saber qué destino eligió el usuario: `createChooser(intent, null, pendingIntent.intentSender)` y leer `EXTRA_CHOOSER_RESULT` en un `BroadcastReceiver`.

6. `onActivityResult` deprecado → `ActivityResultLauncher`

Problema: `startActivityForResult()` y `onActivityResult()` están **deprecados**.

Solución: `registerForActivityResult()` con `ActivityResultLauncher`.

Antes (obsoleto):

```
fun openSecondActivityForResult() {
    val intent = Intent(this, SecondActivity::class.java)
    startActivityForResult(intent, REQUEST_CODE)
}

override fun onActivityResult(requestCode: Int, resultCode: Int, data:
Intent?) {
    super.onActivityResult(requestCode, resultCode, data)
    if (resultCode == Activity.RESULT_OK && requestCode == REQUEST_CODE) {
        // Hacer algo
    }
}
```

Ahora:

```
// Se registra en onCreate u onAttach
val activityResultLauncher =
registerForActivityResult(StartActivityForResult()) { result ->
    // No hay request codes
    if (result.resultCode == RESULT_OK && result.data != null) {
        // Hacer algo
    }
}

fun openSecondActivityForResult() {
    val intent = Intent(this, SecondActivity::class.java)
    activityResultLauncher.launch(intent)
}
```

La actividad que responde no cambia:

```
val intent = Intent()
setResult(FirstActivity.resultCode1, intent)
finish()
```

Ventajas:

1. Se pueden lanzar actividades para resultado **desde fragments** y recibir el resultado en el propio fragment, sin pasar por la activity.
2. Se usan **clases con tipo** en lugar de códigos `Int` propensos a error.
3. **Un callback por petición**, sin necesidad de `switch` / `when` para distinguir resultados.

7. Chuleta de última hora

- View Binding: `viewBinding = true` en `buildFeatures` → `XxxBinding.inflate(layoutInflater)` → `setContentView(binding.root)`.
- En Fragment: `_binding = null` en `onDestroyView()`.
- Adapter: `onCreateViewHolder` (crea, sin datos), `onBindViewHolder` (vincula datos), `getItemCount` (tamaño).
- Explícito = componente concreto. Implícito = acción (`ACTION_VIEW`, `ACTION_SEND`...).
- `setDataAndType()` para URI y MIME a la vez.
- Intent-filter para implícitos: `CATEGORY_DEFAULT` obligatoria. `android:exported` obligatorio en Android 12+.
- `Service` → siempre intent explícito.
- `PendingIntent` en Android 12+ → `FLAG_IMMUTABLE` o `FLAG_MUTABLE`.

- Sharesheet = `Intent.createChooser()`. Sin él = selector del sistema.
- Varias cosas a compartir = `ACTION_SEND_MULTIPLE`.
- Resultado de otra Activity = `registerForActivityResult` + `launch(intent)`. **No** `onActivityResult`.